. **“ STOCK MARKET ANALYZER ”**

A Project Report submitted

In partial fulfillment of the requirements for the award of the degree

of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING (DATA SCIENCE)

Submitted by

BOTTU AKSHAYA LAXMI

Regd No. 2025264835

Under the esteemed guidance of

Dr. Devarapalli Ravi Kishore, Associate Professor, CSE DS

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

GITAM SCHOOL OF TECHNOLOGY

GITAM (Deemed to be University)

VISAKHAPATNAM

2025

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

GITAM SCHOOL OF TECHNOLOGY

GITAM (Deemed to be University)

DECLARATION

I hereby declare that the project report entitled **"STOCK MARKET ANALYZER"** is an original work done in the Department of Computer Science and Engineering (Data Science), GITAM School of Technology, GITAM (Deemed to be University) submitted in partial fulfillment of the requirements for the award of the degree of B.Tech. in Computer Science and Engineering. The work has not been submitted to any other college or University for the award of any degree or diploma.

Date: 06/04/2026

Registration No	Name	Student Signature
2025264835	BOTTU AKSHAYA LAXMI	

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(DATA SCIENCE)

GITAM SCHOOL OF TECHNOLOGY

GITAM (Deemed to be University)

CERTIFICATE

This is to certify that the project report entitled "STOCK MARKET ANALYZER" is a bonafide record of work carried out by BOTTU AKSHAYA LAXMI, Regd No 2025264835, submitted in partial fulfillment of requirement for the award of degree of Bachelors of Technology in Computer Science and Engineering (Data Science).

Date : 06/04/2026

Project Guide

Dr. Devarapalli Ravi Kishore

Associate Professor

TABLE OF CONTENTS

S.No	Description	Page No.
1.	Abstract	5
2.	Introduction	6
3.	Aim of the Project	7
4.	Problem Definition & Objectives	8
5.	Coding	9
6.	Output Screens	19
7.	Conclusion	24
8.	Future Work	25

ABSTRACT

The Stock Market Analyzer Project is a comprehensive data analysis tool designed to bridge the gap between complex financial data and user-friendly insights. Developed using the Python programming language, this application leverages powerful libraries such as yfinance for real-time data retrieval, pandas and numpy for data manipulation, and matplotlib for graphical visualization. The primary objective of this project is to provide users with a menu-driven interface to perform essential financial tasks: analyzing individual stock trends, comparing multiple assets, simulating portfolio growth, and evaluating sector-wide performance. By integrating historical data with automated calculations for volatility and annual returns, the system simplifies the process of market research. Overall, this project serves as a practical application of data science concepts, demonstrating how modular programming and external APIs can be used to build robust financial decision-making tools.

INTRODUCTION

In the modern financial landscape, data is the most valuable asset. The Stock Market Analyzer is a simplified digital platform that replicates the core functions of professional financial analysis software. In an era where retail investing is growing rapidly, providing tools that can automate the fetching and visualization of market data is crucial for informed decision-making. This project utilizes a modular design to ensure clarity and scalability. By using the Yahoo Finance API, the system accesses actual historical market data, ensuring that the simulations and analyses are grounded in reality. The application allows users to create hypothetical portfolios, visualize price movements through line graphs, and rank companies within specific sectors based on their annual performance. By developing this simulation, students gain a practical understanding of:

- API Integration: Fetching live/historical data from external sources.
- Data Visualization: Representing complex numerical trends through graphs and charts.
- Financial Literacy: Understanding concepts like volatility, percentage returns, and asset allocation.
- Modular Programming: Creating a clean, function-based architecture for better maintenance.

AIM OF THE PROJECT

The aim of the Stock Market Analyzer Project is to design and implement a computerized system that provides users with a secure and interactive platform to monitor and analyze stock market trends. This project seeks to empower users to:

- Perform deep dives into individual company performance over specific timeframes.
- Compare different stocks side-by-side to identify the best performing assets.
- Simulate investment strategies through a Portfolio Simulator to see potential growth.
- Analyze sector-specific trends to understand broader market movements.

PROBLEM DEFINITION AND OBJECTIVES

PROBLEM DEFINITION

For many beginner investors, raw financial data is overwhelming and difficult to interpret. Standard banking or trading apps often hide historical trends behind complex interfaces or paywalls. There is a need for a lightweight, transparent tool that allows users to input their own parameters—such as specific years, tickers, and investment amounts—to see clear, visualized results without manual calculation.

OBJECTIVES

MAIN OBJECTIVES:

- To develop a functional, menu-driven Python application for financial analysis.
- To provide a graphical interface using Matplotlib for visualizing stock price history.
- To implement a mathematical engine that accurately calculates returns and volatility.

TECHNICAL OBJECTIVES:

- To utilize Pandas DataFrames for efficient storage and manipulation of time-series data.
- To apply Modular Programming through well-defined functions for each feature.
- To ensure Input Validation so the program handles invalid ticker symbols or incorrect data types gracefully.
- To utilize External Libraries (yfinance) for seamless retrieval of global market data.

CODING

```
# ==============================
# STOCK MARKET ANALYZER PROGRAM
# ==============================

import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt


# ==============================
# Utility Function
# ==============================

def check_good_stock(percentage):
    """Check whether stock is good based on return."""
    if percentage > 10:
        return "Very Good "
    elif percentage > 0:
        return "Good "
    else:
        return "Not Good "
```

```
# ==============================
# 1. Company Stock Analysis
# ==============================

def company_stock_analysis():
    ticker = input("Enter company ticker symbol: ").upper()
    year = input("Enter year (example: 2023): ")

    start_date = f"{year}-01-01"
    end_date = f"{year}-12-31"

    stock = yf.download(ticker, start=start_date, end=end_date)

    if stock.empty:
        print("Invalid ticker or no data available.")
        return

    first_price = stock["Close"].iloc[0].item()
    last_price = stock["Close"].iloc[-1].item()

    percent_change = ((last_price - first_price) / first_price) * 100

    print("\n----- Stock Analysis -----")
    print("Starting Price:", round(first_price, 2))
    print("Closing Price:", round(last_price, 2))
    print("Percentage Change:", round(percent_change, 2), "%")
    print("Stock Performance:", check_good_stock(percent_change))
```

```
    # Plot
    plt.figure()
    plt.plot(stock["Close"])
    plt.title(f"{ticker} Closing Prices ({year})")
    plt.xlabel("Date")
    plt.ylabel("Price")
    plt.grid()
    plt.show()


# ==============================
# 2. Compare Companies
# ==============================

def compare_companies():
    tickers = input("Enter up to 3 ticker symbols separated by space: ").upper().split()

    if len(tickers) == 0 or len(tickers) > 3:
        print("Enter between 1 and 3 tickers.")
        return

    year = input("Enter year (example: 2023): ")
    start_date = f"{year}-01-01"
    end_date = f"{year}-12-31"

    plt.figure()

    returns = {}
```

```
for ticker in tickers:
    stock = yf.download(ticker, start=start_date, end=end_date)

    if stock.empty:
        print(f"No data for {ticker}")
        continue

    first_price = stock["Close"].iloc[0].item()
    last_price = stock["Close"].iloc[-1].item()

    percent_change = ((last_price - first_price) / first_price) * 100
    returns[ticker] = percent_change

    plt.plot(stock["Close"], label=ticker)

if len(returns) == 0:
    return

best_stock = max(returns, key=returns.get)

print("\nAnnual Returns:")
for k, v in returns.items():
    print(k, ":", round(v, 2), "%")

print("\nBest Performing Stock:", best_stock)
print("Performance:", check_good_stock(returns[best_stock]))
```

```
    plt.title("Company Comparison")
    plt.xlabel("Date")
    plt.ylabel("Price")
    plt.legend()
    plt.grid()
    plt.show()


# ==============================
# 3. Portfolio Simulator
# ==============================

def portfolio_simulator():
    try:
        total_investment = float(input("Enter total investment amount: "))
    except ValueError:
        print("Invalid investment amount.")
        return

    tickers = input("Enter ticker symbols separated by space: ").upper().split()

    if len(tickers) == 0:
        print("Please enter at least one ticker.")
        return

    allocations = []
    total_allocation = 0
```

```
# Get allocation %
for ticker in tickers:
    try:
        percent_input = input(f"Enter allocation % for {ticker}: ")
        percent = float(percent_input.replace("%", ""))
    except ValueError:
        print("Invalid percentage entered.")
        return

    allocations.append(percent)
    total_allocation += percent

if total_allocation != 100:
    print("Allocation must sum to 100%")
    return

year = input("Enter year (example: 2023): ")
start_date = f"{year}-01-01"
end_date = f"{year}-12-31"

portfolio_values = None

for i in range(len(tickers)):
    stock = yf.download(tickers[i], start=start_date, end=end_date)

    if stock.empty:
        print(f"No data for {tickers[i]}")
        return
```

```
    # Normalize price
    normalized = stock["Close"] / stock["Close"].iloc[0]

    invested_amount = total_investment * (allocations[i] / 100)
    stock_value = normalized * invested_amount

    if portfolio_values is None:
        portfolio_values = stock_value
    else:
        portfolio_values = portfolio_values.add(stock_value, fill_value=0)

#  FIXED PART
final_value = portfolio_values.iloc[-1].sum()
total_return = ((final_value - total_investment) / total_investment) * 100

print("\n----- Portfolio Result -----")
print("Final Portfolio Value:", round(final_value, 2))
print("Total Return:", round(total_return, 2), "%")
print("Portfolio Performance:", check_good_stock(total_return))

# Plot
plt.figure()
plt.plot(portfolio_values)
plt.title("Portfolio Growth")
plt.xlabel("Date")
plt.ylabel("Portfolio Value")
plt.grid()
```

```
    plt.show()

# ==============================
# 4. Yearly Sector Performance
# ==============================

def sector_performance():

    sectors = {
        1: {"name": "Technology", "stocks": ["AAPL", "MSFT", "GOOGL"]},
        2: {"name": "Automobile", "stocks": ["TSLA", "F", "GM"]},
        3: {"name": "Banking", "stocks": ["JPM", "BAC", "WFC"]}
    }

    print("\nAvailable Sectors:")
    for key in sectors:
        print(key, ".", sectors[key]["name"])

    choice = int(input("Select sector number: "))

    if choice not in sectors:
        print("Invalid choice")
        return

    year = input("Enter year (example: 2023): ")
    start_date = f"{year}-01-01"
    end_date = f"{year}-12-31"
```

```
stocks = sectors[choice]["stocks"]
returns = {}

for ticker in stocks:
    stock = yf.download(ticker, start=start_date, end=end_date)

    if stock.empty:
        continue

    first_price = stock["Close"].iloc[0].item()
    last_price = stock["Close"].iloc[-1].item()

    percent_change = ((last_price - first_price) / first_price) * 100
    returns[ticker] = percent_change

if len(returns) == 0:
    print("No data available.")
    return

sorted_returns = sorted(returns.items(), key=lambda x: x[1], reverse=True)

print("\n----- Ranking -----")
for stock, value in sorted_returns:
    print(stock, ":", round(value, 2), "%")

print("\nBest Performer:", sorted_returns[0][0])
print("Worst Performer:", sorted_returns[-1][0])
```

```
    # Bar Plot
    plt.figure()
    plt.bar(returns.keys(), returns.values())
    plt.title(f"{sectors[choice]['name']} Sector Performance ({year})")
    plt.xlabel("Companies")
    plt.ylabel("Annual Return (%)")
    plt.grid()
    plt.show()


# ==============================
# Main Menu Loop
# ==============================

while True:
    print("\n========== STOCK MARKET ANALYZER ==========")
    print("1. Company Stock Analysis")
    print("2. Compare Companies (up to 3)")
    print("3. Portfolio Simulator")
    print("4. Yearly Sector Performance Analyzer")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        company_stock_analysis()
    elif choice == "2":
        compare_companies()
```

```
elif choice == "3":
    portfolio_simulator()
elif choice == "4":
    sector_performance()
elif choice == "5":
    print("Exiting Program... Thank You!")
    break
else:
    print("Invalid choice. Please try again.")
```

OUTPUT SCREEN

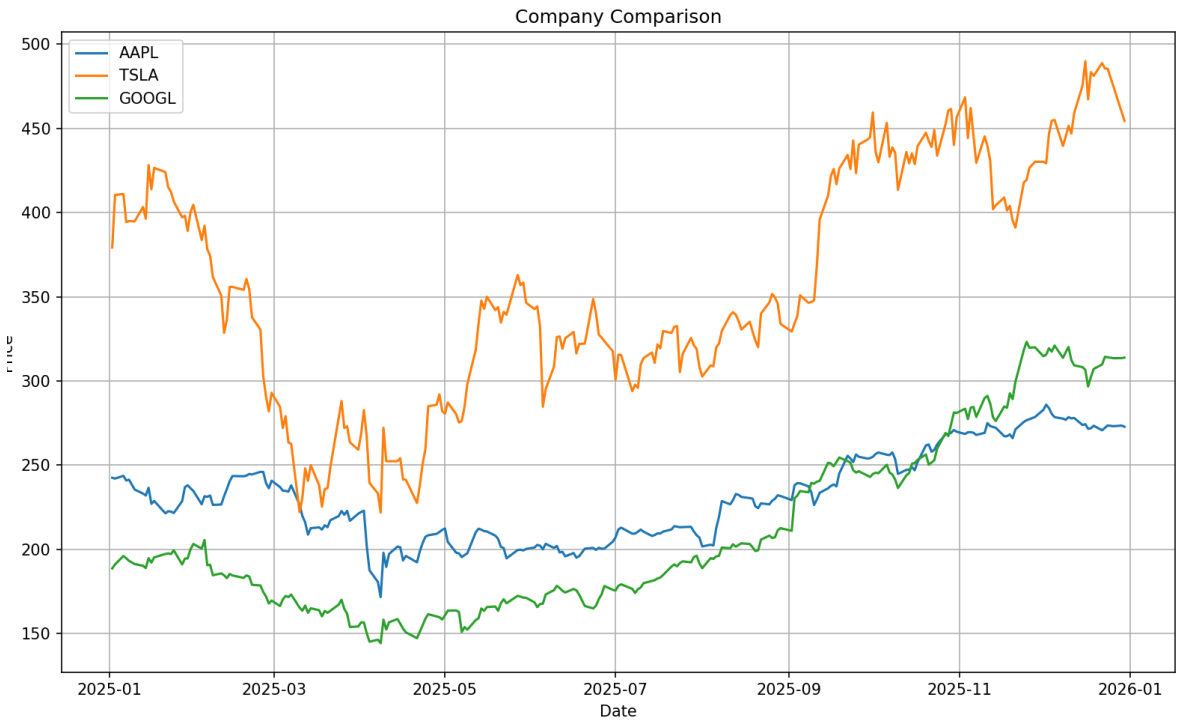
```
========== STOCK MARKET ANALYZER ==========
1. Company Stock Analysis
2. Compare Companies (up to 3)
3. Portfolio Simulator
4. Yearly Sector Performance Analyzer
5. Exit
Enter your choice: ▌
```

1st option :

```
========== STOCK MARKET ANALYZER ==========
1. Company Stock Analysis
2. Compare Companies (up to 3)
3. Portfolio Simulator
4. Yearly Sector Performance Analyzer
5. Exit
Enter your choice: 1
Enter company ticker symbol: AAPL
Enter year (example: 2023): 2025
[*********************100%***********************]  1 of 1 completed

----- Stock Analysis -----
Starting Price: 242.53
Closing Price: 272.82
Percentage Change: 12.49 %
Stock Performance: Very Good 📈
▌
```

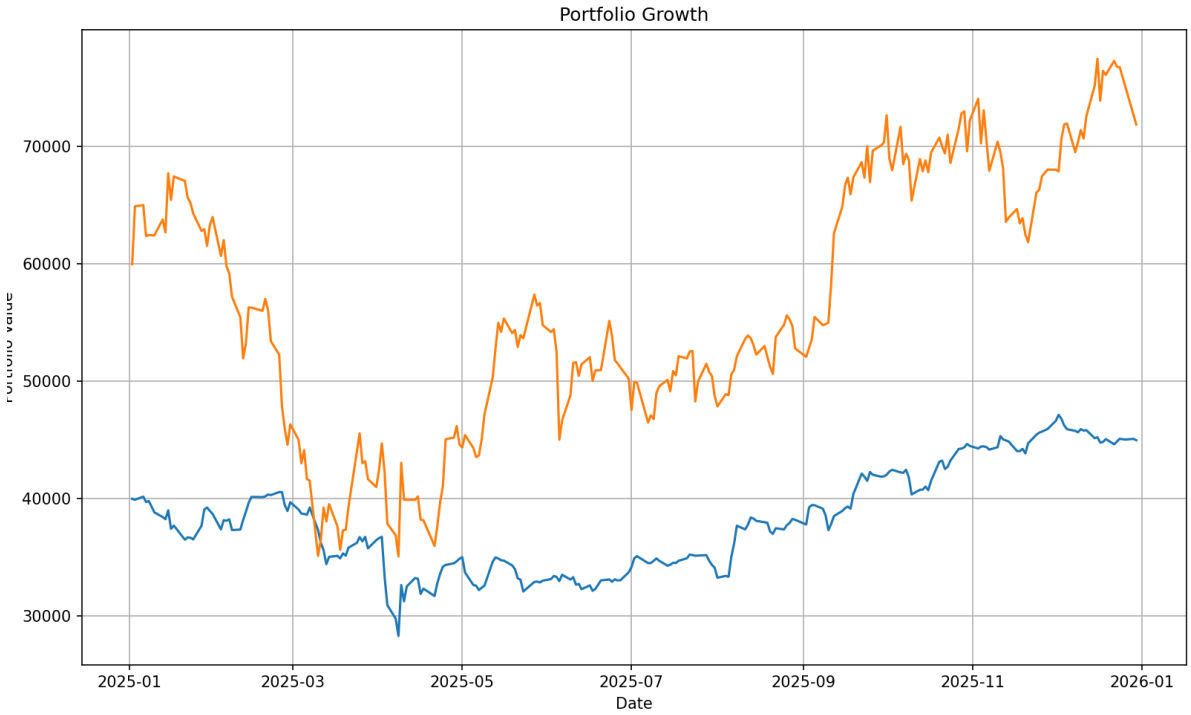
2nd option :

```
========== STOCK MARKET ANALYZER ==========
1. Company Stock Analysis
2. Compare Companies (up to 3)
3. Portfolio Simulator
4. Yearly Sector Performance Analyzer
5. Exit
Enter your choice: 2
Enter up to 3 ticker symbols separated by space: AAPL TSLA GOOGL
Enter year (example: 2023): 2025
[*********************100%***********************]  1 of 1 completed
[*********************100%***********************]  1 of 1 completed
[*********************100%***********************]  1 of 1 completed

Annual Returns:
AAPL : 12.49 %
TSLA : 19.81 %
GOOGL : 66.33 %

Best Performing Stock: GOOGL
Performance: Very Good 📈
```



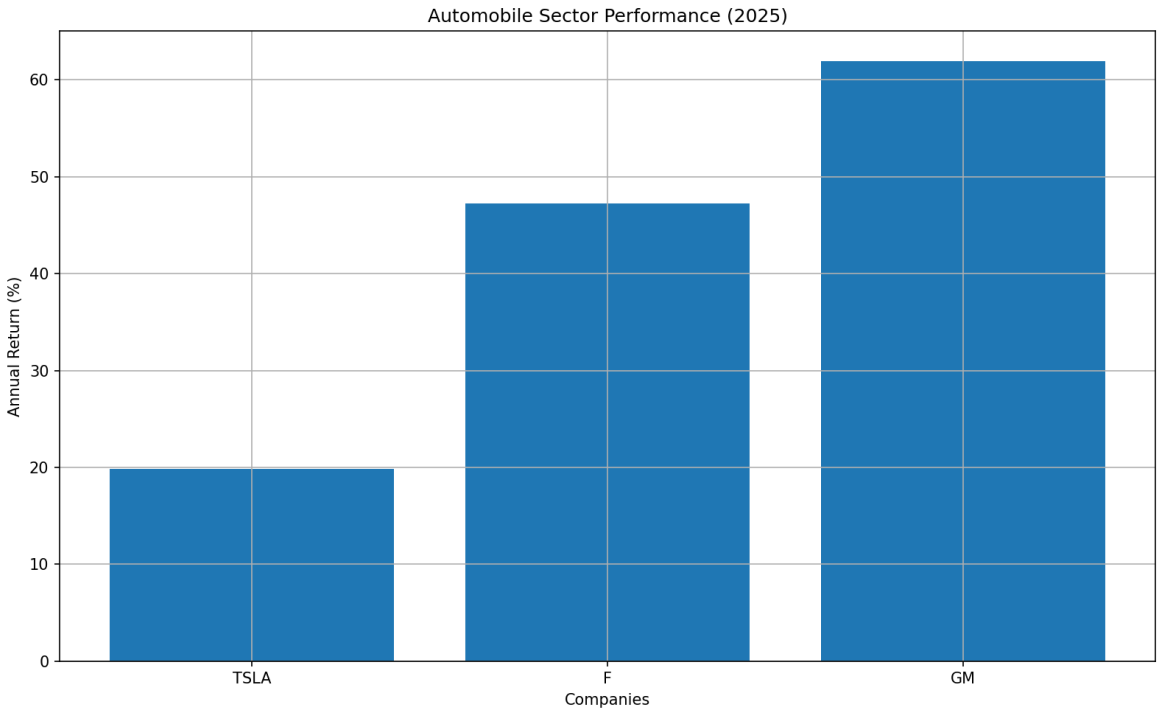
3rd option:

```
========== STOCK MARKET ANALYZER ==========
1. Company Stock Analysis
2. Compare Companies (up to 3)
3. Portfolio Simulator
4. Yearly Sector Performance Analyzer
5. Exit
Enter your choice: 3
Enter total investment amount: 100000
Enter ticker symbols separated by space: AAPL TSLA
Enter allocation % for AAPL: 40
Enter allocation % for TSLA: 60
Enter year (example: 2023): 2025
[*********************100%***********************]  1 of 1 completed
[*********************100%***********************]  1 of 1 completed

----- Portfolio Result -----
Final Portfolio Value: 116885.66
Total Return: 16.89 %
Portfolio Performance: Very Good 📈
```



4th option:

```
========== STOCK MARKET ANALYZER ==========
1. Company Stock Analysis
2. Compare Companies (up to 3)
3. Portfolio Simulator
4. Yearly Sector Performance Analyzer
5. Exit
Enter your choice: 4

Available Sectors:
1 . Technology
2 . Automobile
3 . Banking
Select sector number: 2
Enter year (example: 2023): 2025
[*********************100%***********************]  1 of 1 completed
[*********************100%***********************]  1 of 1 completed
[*********************100%***********************]  1 of 1 completed

----- Ranking -----
GM : 61.93 %
F : 47.26 %
TSLA : 19.81 %

Best Performer: GM
Worst Performer: TSLA
```



5th option:

```
========== STOCK MARKET ANALYZER ==========
1. Company Stock Analysis
2. Compare Companies (up to 3)
3. Portfolio Simulator
4. Yearly Sector Performance Analyzer
5. Exit
Enter your choice: 5
Exiting Program... Thank You!
```

CONCLUSION

The Stock Market Analyzer Project successfully demonstrates the power of Python in simplifying complex financial data. By integrating real-world data through the yfinance API and utilizing the analytical capabilities of Pandas and Matplotlib, the program transforms raw numbers into actionable visual insights.

Throughout the development of this project, several key milestones were achieved:

- Automation: The manual task of searching for historical prices and calculating percentage changes was fully automated.
- Visualization: Data was made accessible through intuitive line graphs and bar charts, making it easier to spot market trends at a glance.
- Modular Efficiency: The use of a menu-driven, function-based architecture ensures that the code is easy to debug, maintain, and expand.

In conclusion, this project serves as an effective educational tool that bridges the gap between programming and finance. It provides a reliable framework for anyone looking to understand market volatility and asset performance, proving that with the right computational tools, data-driven financial decision-making is accessible to everyone.

FUTURE WORK

While the current Stock Market Analyzer provides a solid foundation for financial data analysis, several enhancements could be implemented to increase its utility and depth:

- Real-Time Data Integration: Transitioning from historical daily data to real-time "live" price tracking using WebSockets to provide up-to-the-minute market updates.
- Predictive Analytics (Machine Learning): Implementing models like Long Short-Term Memory (LSTM) or ARIMA to forecast future stock prices based on historical trends.
- Fundamental Analysis Features: Expanding the data retrieval to include balance sheets, P/E ratios, and dividend yields to allow for value-investing analysis.
- Technical Indicators: Adding automated calculations for indicators such as Relative Strength Index (RSI), Moving Averages (SMA/EMA), and Bollinger Bands to assist technical traders.
- User Authentication and Cloud Storage: Moving from a local script to a system with user logins, allowing users to save their portfolios and watchlists in a cloud database (like Firebase or PostgreSQL).
- Graphical User Interface (GUI): Developing a modern desktop interface using Tkinter or PyQt, or a web-based dashboard using Streamlit or Flask.